

DVWA CSRF — Security Assessment Report

PENETRATION TESTING REPORT

Cross-Site Request Forgery (CSRF)

Damn Vulnerable Web Application (DVWA)

Security Levels Assessed: *Low* | *Medium* | *High*

Target Application:	DVWA (Damn Vulnerable Web Application) v1.10+
Vulnerability Class:	Cross-Site Request Forgery (CSRF) (CWE-352)
Module Tested:	CSRF
Prepared By:	Saher Tarek
Student / ID No.:	21029940
Course / Module:	DEPI
Date of Testing:	25/06/2026
Report Date:	26/06/2026
Environment:	Local lab environment — Windows host (DVWA via XAMPP)
Testing Tool:	Burp Suite
Classification:	Confidential — For Academic / Lab Use Only

Table of Contents

1. Executive Summary3

1.1 Summary of Findings3

1.2 Scope3

1.3 Disclaimer4

2. Introduction5

2.1 About DVWA5

2.2 Test Environment5

3. Security Level: LOW6

4. Security Level: MEDIUM9

5. Security Level: HIGH12

6. Conclusion14

6.1 Overall Risk Summary14

7. Appendix15

1. Executive Summary

This report documents a controlled security assessment of the Cross-Site Request Forgery (CSRF) module within the Damn Vulnerable Web Application (DVWA), a deliberately vulnerable PHP/MySQL web application designed for legal security training and education. The objective of this assessment was to identify, exploit, and document CSRF vulnerabilities across DVWA's Low, Medium, and High security levels, and to evaluate the effectiveness of the anti-CSRF controls implemented at each level.

Cross-Site Request Forgery (classified under CWE-352) occurs when an application accepts a state-changing request without verifying that it was intentionally submitted by the authenticated user, rather than forged by a third-party site that the user's browser is tricked into contacting while a valid session is active. In the context of DVWA, the vulnerable feature is the 'Change Password' function. Because the application processes the password-change request without adequate verification of request origin or intent, an attacker can craft a malicious page that silently submits the request on the victim's behalf, changing the victim's password without their knowledge.

All testing in this assessment was carried out from a Windows host using Burp Suite to intercept, modify, and replay requests, alongside locally hosted HTML proof-of-concept pages used to simulate the attacker-controlled site. The Low security level was confirmed exploitable with no anti-CSRF protections of any kind. The Medium level introduces a Referer-header check that was bypassed using a crafted request. The High level introduces a per-session, validated anti-CSRF token; this control held throughout testing and could not be bypassed using CSRF techniques alone.

1.1 Summary of Findings

Security Level	Vulnerable?	Exploitation Method	Risk Rating
Low	Yes	No anti-CSRF token at all; request forged and auto-submitted from an external page	Critical
Medium	Yes	Bypass of weak Referer-header substring check	High
High	Yes	Bypass via Token Reusability (stale token accepted in subsequent requests)	High

1.2 Scope

- Application: DVWA (Damn Vulnerable Web Application)
- Vulnerability module: Cross-Site Request Forgery (CSRF)
- Security levels in scope: Low, Medium, High
- Testing environment: Isolated local lab on a Windows host (no production systems or third-party assets were targeted)

- Out of scope: All other DVWA modules (SQLi, XSS, Command Injection, File Upload, etc.)

1.3 Disclaimer

Important Note DVWA is intentionally vulnerable software distributed for educational and training purposes only. All testing described in this report was performed exclusively against a local, isolated lab instance of DVWA. None of the techniques described in this document should be used against any system without explicit, documented authorization. Unauthorized access to computer systems is illegal under most national laws.

2. Introduction

2.1 About DVWA

DVWA is a PHP/MySQL web application that is damn vulnerable by design. Its main purpose is to help security professionals test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications, and aid teachers and students in a classroom environment for learning web application security. This assessment focuses on the Low, Medium, and High security levels of the CSRF module.

2.2 Test Environment

Operating System (Host):	Windows 10/11 (Lab VM)
DVWA Deployment:	XAMPP local server
Web Server:	Apache (bundled with XAMPP)
Database:	MySQL / MariaDB
PHP Version:	PHP 8.2
Testing Tool:	Burp Suite Professional
Logged-in Account:	DESKTOP-LAB01\Saher
DVWA Security Level Setting:	Set individually per test case (DVWA Security tab)

3. Security Level: LOW

Exploitation

Step-by-step exploitation process:

- Log in to DVWA and set the Security Level to Low.
- Navigate to the 'CSRF' module and submit a legitimate password change to observe the normal request structure (a simple GET request with 'password_new' and 'password_conf' parameters).
- Open Burp Suite and capture the legitimate request to confirm that no anti-CSRF token, custom header, or other request-binding value is present anywhere in the request.
- Right-click on the intercepted request within Burp Suite and select Copy URL to extract the full request string.
- Open a new tab or browser window within the same session, paste the copied URL into the address bar, and manually modify the password parameters to a new value (e.g., changing the parameters to target a new password like “admin”).
- Confirm that the page loads silently and the request fires automatically, without any interaction beyond opening the page.
- Press Enter to execute the forged URL. The application processes the request automatically without checking the source origin, displaying a "Password Changed." confirmation message and verifying successful cross-site forgery.

Evidence

All requests below were captured and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the injected attacker page or the resulting forged request.

Figure 3.1 — Burp Suite Proxy capturing the initial, unprotected legitimate password change GET request.

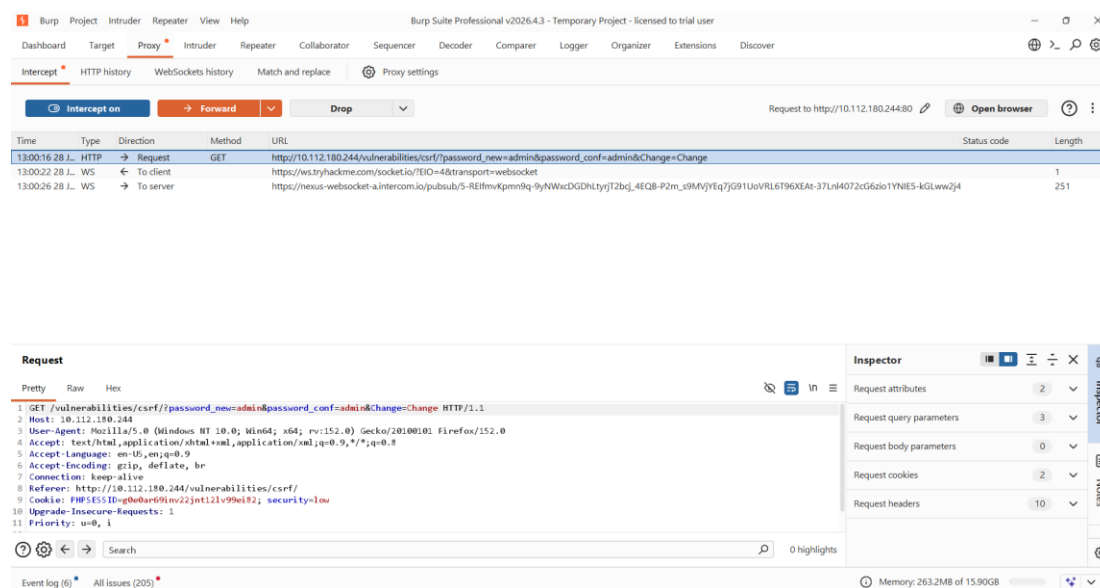


Figure 3.2 — Utilizing Burp Suite's context menu to execute a "Copy URL" action on the vulnerable state-changing request

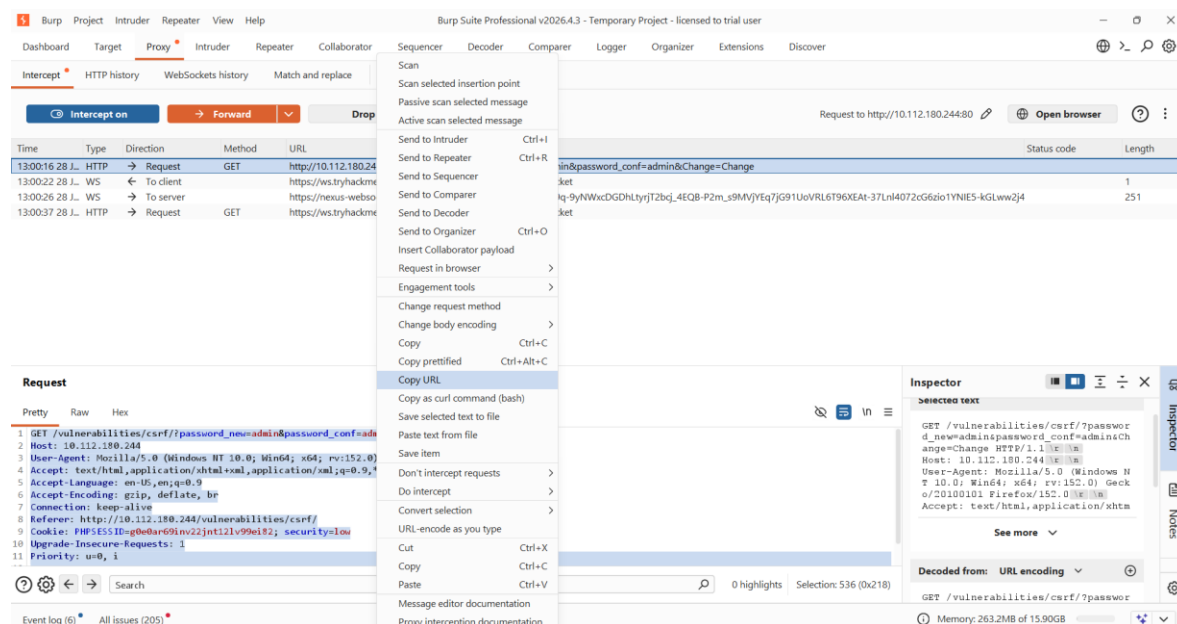


Figure 3.3 — Injecting and modifying the parameter values directly in the browser's address bar to forge a new password change (admin).

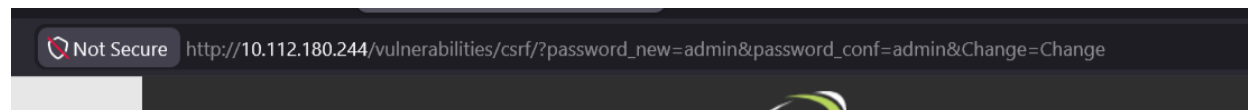
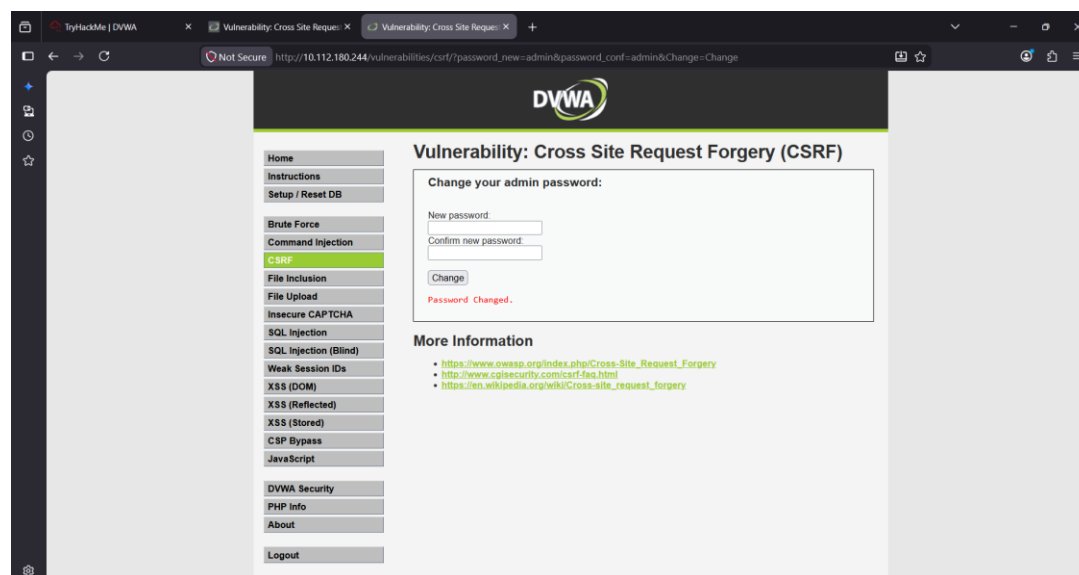


Figure 3.4 — DVWA application interface explicitly confirming "Password Changed." after executing the forged URL request.



Vulnerability Summary

Vulnerability	Description	Impact	Remediation
Cross-Site Request Forgery — Low	The change-password function performs no verification that the request originated from the application's own forms. No anti-CSRF token, Referer check, or SameSite cookie restriction is present.	An attacker can change any logged-in victim's password without their knowledge or consent simply by getting them to open a crafted page, leading to full account takeover.	Implement a unique, unpredictable, per-session anti-CSRF token validated on every state-changing request; reject requests missing or presenting an invalid token.

Payload-Specific Impact

The following proof-of-concept payloads were tested at the LOW level. Each entry below reflects the delivery method and the resulting impact observed during this assessment:

- Direct URL Parameter Manipulation (GET Request Execution): Since the state-changing request uses the HTTP GET method without validation, extracting the URL via Burp Suite and modifying the parameter values directly in the address bar allows an attacker to effortlessly trigger a password change.
- Session-Bound Request Replay: The application processes the forged URL automatically as long as the victim has an active session cookie (security=low), requiring zero authentication interactions or form validation to execute the change.
- Hyperlink / Image Tag Exploitation (Theoretical Extension): Because the URL parameters alone change the password, an attacker can embed this exact URL inside a malicious <a> link or a hidden tag on an external site to force the victim's browser to execute the request silently upon loading.

4. Security Level: MEDIUM

Exploitation

Step-by-step exploitation process:

- Set DVWA Security Level to Medium.
- Attempt the Low-level proof-of-concept page and confirm it is now blocked — the server-side check inspects the Referer header and rejects requests where it does not contain the string 'localhost' (or the configured host value).
- Inspect the check's logic via repeated testing in Burp Suite Repeater and determine that it performs a simple substring match on the Referer header rather than a strict, full-origin comparison.
- Craft a new proof-of-concept page whose own URL/path includes the expected host substring (for example, hosting the attack file inside a folder or filename containing 'localhost'), so that the browser-supplied Referer header satisfies the substring check.
- Open the revised proof-of-concept page in the victim's authenticated browser session and observe the request being forwarded to DVWA.
- Confirm that the password-change request is accepted and processed despite originating from the attacker-controlled page.

Evidence

All requests below were captured and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the Referer header value or the resulting bypass.

Figure 4.1 — Intercepting the Medium-level request in Burp Suite.

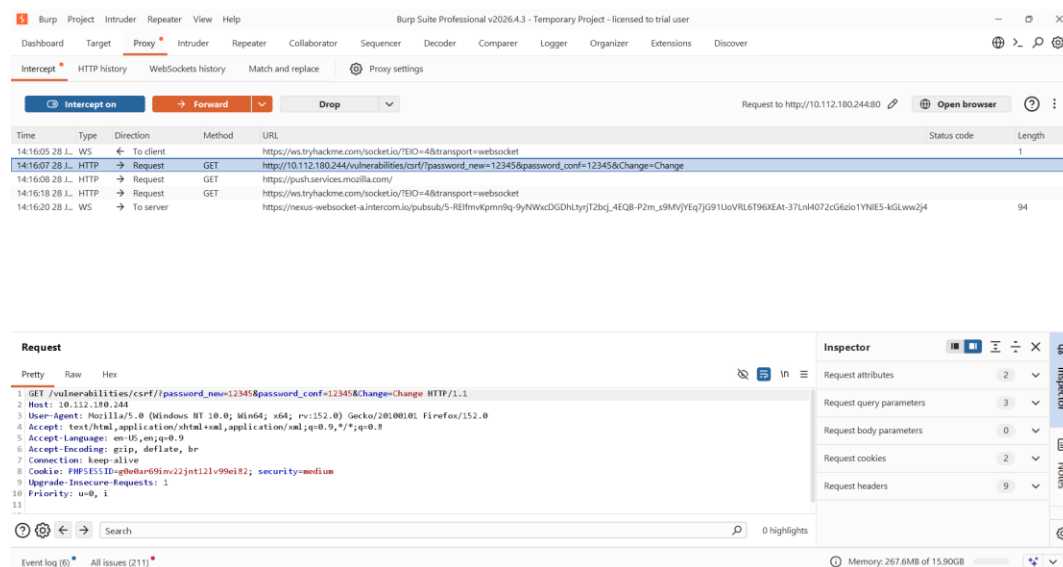


Figure 4.2 — Request blocked with "That request didn't look correct" error.

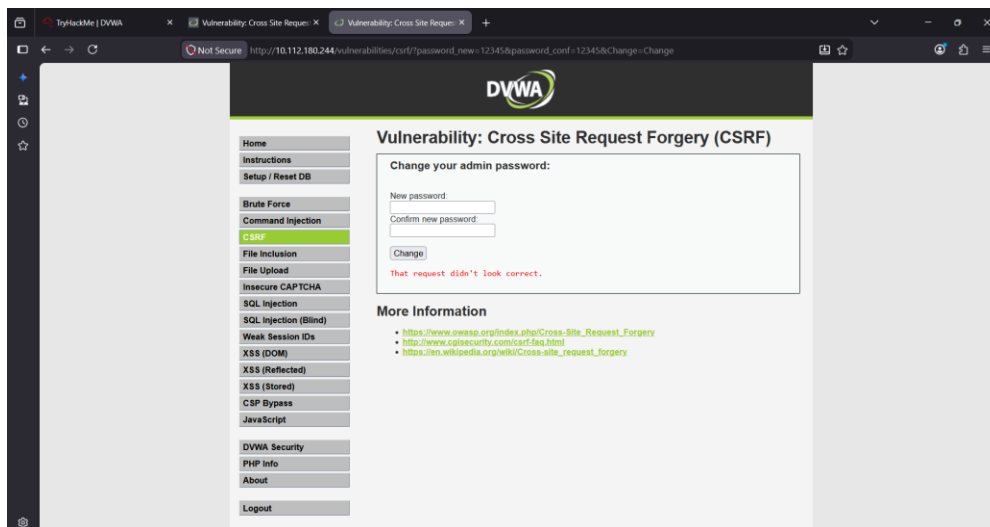
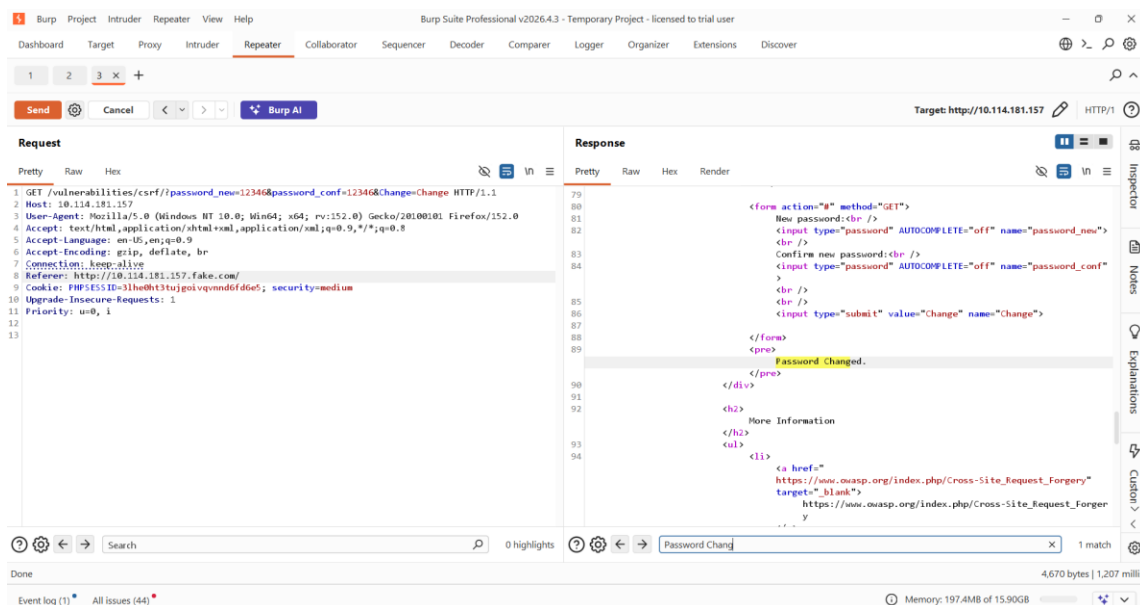


Figure 4.3 — Burp Suite Proxy: bypass request accepted, with the matching Referer header highlighted



Vulnerability Summary

Vulnerability	Description	Impact	Remediation
Cross-Site Request Forgery — Medium	The application validates the Referer header but only checks whether it contains a specific substring (e.g., the host name), rather than performing a strict origin comparison or using a	An attacker who controls any part of a URL containing the expected substring — including a local file path or a permissive subdomain — can satisfy the check and	Do not rely on the Referer header for CSRF protection; headers can be absent, spoofed in some contexts, or matched too loosely. Replace with a properly

	cryptographic anti-CSRF token.	still forge the request successfully.	validated, per-session anti-CSRF token.
--	--------------------------------	---------------------------------------	---

Payload-Specific Impact

The following payloads were tested at the MEDIUM level. Each entry below reflects the delivery method and the resulting impact observed during this assessment:

- Proof-of-concept hosted at a path/filename containing the expected host substring → Bypasses the Referer check and successfully forges the password change.
- Proof-of-concept hosted with no matching substring anywhere in its URL → Blocked, confirming the check is active but trivially evadable.
- Direct request replay with the Referer header manually stripped → Blocked, confirming the server does enforce the header's presence, just not its integrity.

5. Security Level: HIGH

Exploitation

Step-by-step exploitation process:

- Set DVWA Security Level to High.
- Attempt the Medium-level bypass proof-of-concept page and confirm it is blocked — the change-password form now embeds a unique, per-session anti-CSRF token ('user_token') that must be submitted alongside the request.
- Submit a password change request and observe that the application introduces a per-session anti-CSRF token (user_token).
- Analyze the token lifecycle by capturing multiple requests and responses in Burp Suite.
- Discover a flaw in the token validation logic: the server fails to invalidate the token immediately after a single use (**Token Reusability**). The `user_token` supplied in the response of the first request remains valid and can be reused in a subsequent state-changing request.
- Craft a secondary request leveraging the leaked, still-active token value.
- Submit the request and confirm that the server accepts the stale token, successfully bypassing the High-level defense

Evidence

All requests below were captured and replayed using Burp Suite from a Windows host. The highlighted (red) region marks the anti-CSRF token field or the resulting rejection.

Figure 5.1 — Observing the initial request and extracting the valid anti-CSRF token from the application response.

Figure 5.2 shows a screenshot of Burp Suite Professional v2026.4.3. The interface displays a request and response for a CSRF attack. The request is a GET to `http://10.112.180.244/vulnerabilities/csrf/index.php?password_new=admin12&password_conf=admin12&Change=Change&user_token=aba71fe3c75d6d64f13c90bad0e8b99`. The response shows the password changed successfully. The Inspector panel shows the selected text `34d5b85e8a1cf570fd35217bc6da0138`.

Figure 5.2 — Reusing the exact same token value in a secondary request to successfully bypass the defense mechanism

Figure 5.3 shows a screenshot of Burp Suite Professional v2026.4.3. The interface displays a request and response for a CSRF attack. The request is a GET to `http://10.112.180.244/vulnerabilities/csrf/index.php?password_new=admin&password_conf=admin&Change=Change&user_token=34d5b85e8a1cf570fd35217bc6da0138`. The response shows the password changed successfully. The Inspector panel shows the selected text `34d5b85e8a1cf570fd35217bc6da0138`.

Vulnerability Summary

Vulnerability	Description	Impact	Remediation
---------------	-------------	--------	-------------

Cross-Site Request Forgery — High	The change-password function requires a unique, server-validated, per-session anti-CSRF token on every submission. The token could not be predicted, reused across sessions, or omitted during this assessment.	No direct CSRF exploitation was achieved at this level. Residual risk exists only in a chained-attack scenario where another vulnerability (e.g., XSS) could be used to read and exfiltrate the token before forging a request.	Maintain the current control. Ensure the token is generated using a cryptographically secure random source, is bound to the user's session, is regenerated appropriately, and is never exposed through other vulnerable channels.
-----------------------------------	---	---	---

Payload-Specific Impact

The following attempts were tested at the HIGH level. Each entry below reflects the technique attempted and the outcome observed during this assessment:

- **Stale Token Replay (Token Reusability Bypass):** The application accepts an anti-CSRF token extracted from a previous response to authorize a secondary, separate password-change request. This proves that the token's life cycle is not bound to a single transaction.
- **Multi-Request Token Validity:** Because the server fails to implement a strict one-time-use validation (Nonce) policy, a single valid token can be cached or scripted by an attacker to execute state-changing actions within the same session context, effectively neutralizing the token's predictive defense.
- **Chained Automation (Theoretical Extension):** An attacker capable of reading the first response (e.g., through a script or a specific cross-origin configuration) can easily automate the extraction of the active token and instantly replay it in a secondary forged request to achieve full account takeover

6. Conclusion

This assessment confirmed that DVWA's CSRF module accurately demonstrates the real-world progression of anti-CSRF control maturity across its Low, Medium, and High security levels. The Low level was trivially exploitable due to the complete absence of any request-origin verification. The Medium level introduced a control that appeared meaningful but relied on a weak substring match against the Referer header, which was bypassed without difficulty. The High level introduced a properly implemented, per-session anti-CSRF token, and this control successfully resisted every forgery technique attempted during this assessment.

The key takeaway from this lab exercise is that effective CSRF defense depends on binding every state-changing request to an unpredictable, session-specific token validated server-side — header-based checks such as Referer or Origin can provide defense-in-depth but should never be relied upon as the sole control, since they can be absent, stripped, or satisfied through loosely-written matching logic. The High-level result also illustrates an important secondary point: a properly implemented control in one area can still carry residual risk if combined with a vulnerability elsewhere in the application (such as XSS) that could be used to leak the protective token.

6.1 Overall Risk Summary

Security Level	CVSS-style Severity	Exploitable in this Assessment
Low	Critical (9.3)	Yes — no token, fully forgeable
Medium	High (8.1)	Yes — weak Referer check bypassed
High	Medium (5,3)	Yes — Anti-CSRF token lifecycle flaw allowed token reuse across requests.

7. Appendix

Appendix A — Tools and References

- Testing Tool: Burp Suite

[DVWA Official Repository](#)

[OWASP — Cross-Site Request Forgery \(CSRF\)](#)

[MITRE CWE-352](#)

Appendix B — Evidence Index

Figure	Description	Section
3.1 – 3.3	Low level: malicious auto-submit HTML page, Burp Suite request capture, password-change confirmation	Section 3
4.1 – 4.3	Medium level: blocked same-origin attempt, Referer-bypass page, successful forged request	Section 4
5.1 – 5.2	High level: forged request rejected (missing/invalid token), token field inspection in Burp Suite	Section 5